
Real-World PBTs as Lean Challenges

Anonymous Author(s)

Affiliation

Address

email

Abstract

We present a benchmark for evaluating AI models on real-world formal software verification tasks. We first scrape $\approx 20k$ property-based tests (PBTs) from permissively licensed real-world Python repositories, then automatically translate them into Lean 4 specifications with `sorry` placeholders. Translating PBTs into formal specifications is challenging: it requires modeling Python semantics in Lean, bridging the gap between input/output testing and structural proofs, and handling the inherent difficulties of dependently-typed programming. We describe a three-agent LLM pipeline for transpiling PBT suites into Lean specifications, evaluate coverage and quality metrics, and provide baselines for proof completion using several automated and model-based approaches. Our benchmark aims to drive progress on the underexplored problem of AI-assisted formal verification of real-world software, which we claim is of increasing import as AI begins to produce an unprecedented percentage of the world’s code.

1 Introduction

Several AI safety proposals are premised on AI-driven formal verification (FV); this is a core hypothesis of the ARIA Safeguarding AI program and the related Guaranteed Safe AI agenda Dalrymple et al. [2024]. If FV is to play this role, it must scale to the complexities of real-world code, but today we have little evidence that such capabilities are possible. Relevant benchmarks are mostly small datasets crafted by hand Loughridge et al. [2024], larger datasets extracted from expert-driven verification projects Chakraborty et al. [2025], or are focused on advanced mathematics rather than engineering Glazer et al. [2024]. None of these are likely to be representative of the formal verification tasks required to verify real-world AI-generated software at scale.

We generate a new benchmark, FVSpec, from public uses of property-based testing (PBT). While FV itself is rarely used in software development, PBT is a ‘close sibling’ technology which has seen significant adoption. In both FV and PBT, engineers write *specifications*: logical properties that the software must obey. For example, we might require that a Python function `isort(1st)` always returns a sorted permutation of its input `1st`. The difference lies in how this property is checked. In FV, the property is proved using mathematical techniques—this results in almost-perfect confidence, but requires proof engineering by highly expert teams Dodds [2022]. In PBT, the code is randomly tested—e.g. for random values of `x`. This is a push-button process with no proof engineering.

PBT is typically much cheaper than FV, and as a result has seen wider adoption. In fact, PBTs are typically seen as the first step toward FV. For example, in the interactive theorem prover ACL2s, any unproven theorem is automatically treated as a PBT (and tested against randomly generated inputs) until the author manages to supply a proof. Thus the idea of translating PBTs to theorems for FV is natural and, to an extent, industry-standard. We build on this intuition. First, we scrape $\approx 20k$ permissively-licensed PBTs written in Python’s Hypothesis framework MacIver et al. [2019]. Each PBT in our dataset can be seen as a small theorem statement (or *ansatz*) about a piece of Python code, which the author would like to prove or, failing that, at least test. To make it possible for an AI to

prove or refute these theorems, we then translate both code and PBTs to the theorem prover Lean, which is becoming the de-facto standard in AI formal verification. This approach was pioneered by Dougherty and Mehta on the FVAPPS verification benchmark Dougherty and Mehta [2025]. In FVAPPS, source programs were taken from the APPS benchmark by Hendrycks et al Hendrycks et al. [2021], translated to PBTs, and then lifted to theorems. We apply the same process at scale to PBTs found in the wild.

We use AI-driven translation to generate Lean programs and theorems at scale. LLMs are quite good at program translation, and FVAPPS has shown that this approach is practical for constructing Lean theorems. Although in some cases the translations may be imperfect, lessening the importance of any proof or refutation of the resultant theorem from the perspective of the author of the originating PBT, it is nevertheless the case that we produce high-quality and realistic FV challenges for AI benchmarking. The result is tens of thousands of verification challenges, each consisting of (1) a program and (2) a desired specification, both written in Lean. Unlike existing FV benchmarks, each problem corresponds to properties of real-world software written by engineers with no guaranteed formal verification experience.

TODO: Summary of results. The rest of the paper is organized as follows. We formalize the task in Section 2. We describe our generation pipeline—dependency extraction, LLM-based transpilation with iterative Lean LSP repair, and post-production cleaning—in Section 3. We characterize our input corpus, detailing how we scraped and deduplicated $\approx 20k$ permissively-licensed Hypothesis tests from open-source repositories, in Section 4; we also compare against the concurrently released Hypothesis Corpus 2026 there. We present results in Section 5, discuss threats to validity in Section 6, survey related work in Section 7, and conclude in Section 8.

2 Task Definition

Each problem in the FVSpec benchmark consists of three components: (1) the original Python property-based test, (2) a Lean theorem corresponding to the PBT, with a sorry placeholder where the proof should go, and (3) a Lean implementation of the functions referenced in the theorem. The AI’s task is to complete the proof by replacing sorry with a valid Lean proof term. Each problem also includes a URL pointing to the original source code and license information.

2.1 Running Example

Consider a Hypothesis test asserting the Pythagorean trigonometric identity $\sin^2(x) + \cos^2(x) = 1$, for a trigonometric library `trg`:

```

71 from hypothesis import given, strategies
72
73 @given(x=strategies.floats(
74     min_value=-sys.float_info.max / 4,
75     max_value=sys.float_info.max / 4,
76     allow_infinity=False))
77 def test_property(x):
78     assert trg.sine(x) ** 2 + trg.cosine(x) ** 2 == pytest.approx(1)

```

Our pipeline translates this into a Lean implementation:

```

81 namespace Fvspec.Impl
82
83 def to_radians (degrees : Float) : Float :=
84     degrees * Float.pi / 180.0
85
86 def sine (theta : Float) : Float :=
87     Float.sin (to_radians theta)
88
89 end Fvspec.Impl
90

```

And a corresponding Lean specification:

```

93 import Fvspec.Impl
94

```

```

95 open Fvspec.Impl
96
97 def cosine (theta : Float) : Float := sorry
98
99 theorem sine_cosine_pythagorean (x : Float) :
100   (sine x) * (sine x) + (cosine x) * (cosine x) = 1 := sorry

```

102 The AI must fill in both sorry placeholders with valid Lean proof terms.

103 **TODO**– Explain how we do not faithfully translate the original `trg` library into Lean, with all
104 dependencies and so forth, rather, we utilize Lean’s built-in `sine` and `cosine` functions. This is
105 representative of our general attitude which is that our primary goal is to produce equally interesting,
106 realistic problems to benchmark AI on, while actually proving properties about this real-world
107 software to help these specific code authors is just a secondary goal.

108 2.2 PBT vs. Proof

109 In PBT, a property is checked by randomly generating inputs and testing them—the process is push-
110 button but provides only statistical confidence. In FV, the property is proved using computer-aided
111 symbolic logic, providing near-perfect confidence but requiring proof engineering.

112 Translating a PBT into a formal specification thus involves a shift from input/output testing to proofs
113 over the structure of the program. Our pipeline filters nonsensical theorems, but any discrepancies
114 that remain will not affect the usefulness of the dataset. Because the Lean theorem prover provides a
115 ‘perfect’ oracle, every theorem we generate is a useful challenge problem, even if it differs from the
116 original PBT. In fact, some PBTs in our input dataset will be naturally false due to undetected edge
117 cases.

118 **TODO**- Explain what we mean by nonsensical theorems.

119 **TODO**- Explain to what degree Lean can be viewed as a perfect oracle.

120 2.3 Challenges

121 Formal verification is unusually difficult to benchmark for two reasons. First, just as with advanced
122 mathematics, formal verification tasks are highly varied, and constructing interesting problems
123 requires considerable expertise. FV tasks also vary in difficulty—they can be as shallow as checking
124 the type-safety of simple functions, or as deep as proving an unsolved mathematical theorem such as
125 the Collatz Conjecture. Our benchmark avoids the need to select and hand-build challenge problems
126 by translating naturally occurring PBT problems written by real-world engineers.

127 Second, formal verification is little-used outside academia. As a result, there are few natural datasets,
128 and those that exist are fragmented between multiple FV tools (e.g. SAW/Cryptol for cryptographic
129 systems, ACL2/Coq for traditional software verification, Lean for mathematics, SPIN/TLA+ for
130 protocol analysis)—or are privately held, closely guarded corporate IP (such as the FV work done by
131 companies like Intel or Synopsys). Public FV examples are often pedagogical exercises for which
132 the proof exists in many different forms in the training set. Our benchmark solves this problem
133 by translating from a ‘close sibling’ domain, PBT, where many more datapoints exist outside of
134 pedagogy or academic research. As a result, most theorems in our benchmark will have never been
135 verified by anyone.

136 **TODO**- A reviewer will ask us to substantiate this claim.

137 Beyond these structural challenges, AI-driven translation at scale introduces practical difficulties.
138 Each failed pipeline run is expensive—a single sample may require multiple LLM calls with iterative
139 repair, and a bug in the pipeline infrastructure can invalidate an entire batch. Quality improvements
140 must therefore be targeted: we identify specific failure modes in the output (e.g. type restrictions,
141 missing dependencies, trivial stub theorems) and address them incrementally rather than re-running
142 the full dataset. We initially considered extending the benchmark to TypeScript PBTs, but found that
143 the PBT ecosystem in TypeScript is far less mature, and the additional coverage would not justify the
144 engineering effort.

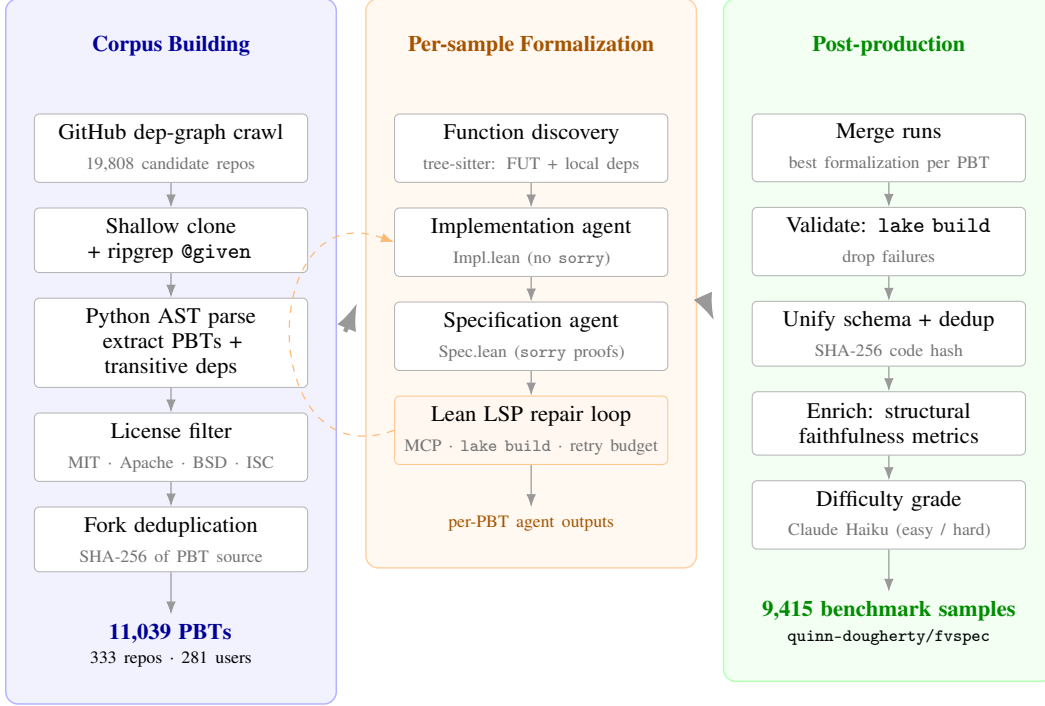


Figure 1: FVSpec end-to-end pipeline. *Left*: the RealPBT corpus is assembled by crawling the Hypothesis GitHub dependency graph, cloning repositories, extracting `@given`-decorated PBTs via AST parsing, and deduplicating at both the fork and code level. *Centre*: each PBT is formalized by an LLM agent producing a computable Lean implementation (`Impl.lean`) and a theorem specification (`Spec.lean`); all agents operate within an iterative Lean LSP repair loop. *Right*: raw outputs are merged, compiled, deduplicated, enriched with structural-faithfulness metrics, and difficulty-graded.

145 3 Pipeline

146 Our generation pipeline is modelled on FVAPPS Dougherty and Mehta [2025], but starts from real-
 147 world Hypothesis property-based tests (PBTs) rather than synthetic programming puzzles. As shown
 148 in Figure 1, the pipeline has three phases: (i) corpus building, which crawls the Hypothesis GitHub
 149 dependency graph, extracts PBTs via AST parsing, and deduplicates at the fork and code level;
 150 (ii) per-sample formalization, which recovers the function under test and dependencies, translates
 151 them to computable Lean, and generates `sorry`-carrying theorem specifications via an iterative
 152 LSP repair loop; and (iii) post-production, which merges, validates, deduplicates, enriches with
 153 structural-faithfulness metrics, and difficulty-grades the raw agent outputs.

154 3.1 Input PBT corpus

155 Source PBTs are drawn from our RealPBT dataset (citation omitted for double-blind review);
 156 Section 4 describes how the corpus was scraped, fork-deduplicated, and cleaned in detail. The
 157 working set after deduplication is 11,039 PBTs across 333 repositories.

158 3.2 Function discovery

159 For each PBT, we use `tree-sitter` to extract the Python source code of the FUT and any locally-defined
 160 dependencies. This function discovery step succeeds for **TODO** % of samples; when it fails, we
 161 generate a stub FUT with generic type parameters to avoid producing trivial theorems.

162 3.3 LLM-based transpilation

163 We translate each sample using three agents.

164 **Implementation agent** The implementation agent runs first, translating the FUT and its dependen-
165 cies into fully computable Lean definitions. No sorry placeholders are permitted, so the output must
166 typecheck and evaluate. Dependencies are formalized separately and then merged with deduplica-
167 tion into a single `Impl.lean` file. On success, we additionally extract Lean type signatures for all
168 formalized functions and provide them as context to downstream agents. If the agent exhausts its
169 retry budget without producing typechecking code, downstream stages still run, but without a verified
170 implementation to build on.

171 **Specification and units agents** Next, the specification agent and units agent run in parallel.
172 The specification agent translates Hypothesis tests into Lean theorems that import the formalized
173 implementation; proofs are left as sorry. The units agent translates any available Python unit tests
174 into Lean/LSpec checks. The units output is included as auxiliary validation, but does not gate
175 inclusion of the specification (**TODO** confirm policy).

176 3.4 Iterative repair against Lean

177 All agents operate in an iterative repair loop. At each step, we check the generated Lean code against
178 the compiler via the Lean LSP (accessed through an MCP tool server). If the code does not typecheck,
179 we feed the compiler diagnostics back to the agent and prompt for a revised translation. This loop
180 continues until the code typechecks or a retry budget is exhausted. We validate final compilation with
181 a full `lake build`.

182 3.5 Cleanup and quality evaluation

183 After the formalization agent produces Lean artifacts, a multi-stage post-production pipeline processes
184 the raw run outputs into a final curated dataset. The stages are: `merge` $\rightarrow$ `validate` $\rightarrow$ `unify` $\rightarrow$ `dedup`
185 $\rightarrow$ `enrich` $\rightarrow$ `grade` $\rightarrow$ `metrics`. Each stage is an independent CLI command (`uv run postprod`
186 `<stage>`) and the full pipeline state is checkpointed so it can be resumed after interruption.

187 **Merge.** Multiple independent benchmark runs (identified by timestamped run directories) are
188 merged into a single JSONL file. Each sample directory contains `datapoint.json` (source PBT
189 metadata), `qa.json` (agent outputs and metrics), and `Spec.lean/Impl.lean` (generated Lean
190 artifacts). When the same `sample_id` appears in multiple runs, the merge step keeps the highest-
191 quality copy, scored by: (1) success flag, (2) overall structural faithfulness, (3) theorem count,
192 (4) presence of unit tests, (5) implementation autoformalization success, (6) specification signature
193 success. Git commit hash, model, run timestamp, and Lean toolchain version are recorded as
194 provenance fields.

195 **Validate.** Because an agent can produce code that typechecks in isolation but fails under a full
196 project build, we validate compiled samples by instantiating each `Spec.lean/Impl.lean` pair into a
197 `lake build` project and running the Lean compiler. Validation runs up to 400 samples by default
198 (yielding $\pm 5\%$ accuracy at 95% CI) or all samples with `-all`. Only samples that pass `lake build`
199 are retained in the dataset; failures are logged but dropped.

200 **Unify.** Our dataset was produced in two pipeline generations (*feb03* and *apr08*) with
201 schema drift between them. The unify step resolves these differences: it renames `radon`
202 $\rightarrow$ `realpbt_radon`, backfills the `structured_dependencies` list from feb03’s legacy par-
203 allel JSON-stringified arrays (filling null for fields not captured in that schema), drops
204 100%-null dead fields (`lean_metrics.tests.structure`, `lean_metrics.tests.complexity`,
205 `metrics_metadata.tests.available`), and reassigns fresh monotone `sample_ids` 1..N with
206 feb03 rows first. Each row carries a run tag indicating its source generation.

207 **Code-level deduplication.** The same PBT may have been formalized in multiple runs (e.g., because
208 sampling overlap exists between run index ranges, or because a PBT appeared in both feb03 and
209 apr08). This step groups samples by SHA-256 of `realpbt_code` and keeps the single best repre-
210 sentative per group (using the same quality scoring as the merge step), separately tracking cross-run
211 vs. within-run duplicates for audit.

Enrich. Every sample is annotated with: `code_hash` (SHA-256 of `realpbt_code`), `is_canonical` (whether it is the best formalization of its PBT), `formalization_rank` (1-based rank within the group, 1 = canonical), `num_formalizations` (how many runs produced a formalization of this PBT), and `pareto_dominated` (whether a strictly better sample exists on all five structural faithfulness sub-metrics: parameter coverage, type correspondence, strategy coverage, assertion coverage, and dependency coverage). Stale v1 grader fields with inconsistent coverage are also stripped here, and columns are reordered for clean HuggingFace presentation.

Difficulty grading. Each sample is scored for formalization difficulty by Claude Haiku, producing a binary easy/hard label (`difficulty_binary`) with a calibrated confidence score and a brief reasoning trace. The grader uses a structured prompt including the PBT source code, the generated Lean spec, and contextual information about the formalization strategy. Grading is resumable: a checkpoint file accumulates completed grades so a killed run restarts where it left off.

Lean metrics. A final metrics pass runs `radon` on the Python source and a custom Lean AST analyzer on `Spec.lean/Impl.lean` to populate `lean_metrics` (total lines, sorry count, axiom count, spec structure and complexity fields including Halstead volume and difficulty, theorem/def counts).

The resulting specification/implementation pairs are then scored and filtered. We compute *structural faithfulness* programmatically by comparing the generated Lean artifacts to the source PBTs. The metric decomposes into sub-scores for parameter coverage, type correspondence, strategy coverage, assertion coverage, and dependency coverage, producing an overall score (**TODO** define weights; add exact formula). We also assign a *difficulty* rating using an LLM judge (**TODO** specify model and prompt), intended to approximate the proof effort required on a 1–10 scale.

4 Dataset Characterization

4.1 Input corpus: scraping and deduplication

Scraping. Source PBTs come from our RealPBT dataset (citation omitted for double-blind review)¹, assembled by a custom Python scraper that operates on the GitHub dependency graph of the Hypothesis library. The scraper first built the candidate repository list by crawling the GitHub dependency graph of the Hypothesis library HypothesisWorks [2024] page by page using the PyGitHub API, collecting 19,808 unique owner/repository pairs into a static snapshot.² For each candidate, the scraper iterates asynchronously with rate-limit-aware GitHub REST API backoff, shallow-clones the repository, and uses `ripgrep` to locate files containing the `@given` decorator. For each such file, it parses the Python AST to extract all PBT functions—identified as function definitions whose decorator list includes a call to `given`—and performs a transitive static call-graph traversal to recover the minimal closure of locally-defined helpers called by each PBT. Repository metadata (name, URL, license, stars, forks, commit hash) and extracted PBTs (source code, dependency names and their source) are persisted to PostgreSQL; repos whose commit hash was already processed are skipped, enabling incremental re-scraping. Only repositories whose license matches a permissive allowlist (MIT, Apache-2.0, BSD variants, ISC, Unlicense, CC0, BlueOak-1.0.0) contribute PBTs to the released dataset.

Fork deduplication. The raw scrape, run with a 24-hour time budget, yielded 54,345 PBTs, but GitHub’s dependency graph surfaces many forks of the same upstream (e.g., hundreds of forks of `pytorch` each carrying an identical Hypothesis test suite). We deduplicate by grouping repositories whose names share the same base component, selecting a canonical representative per group (highest star count, breaking ties by PBT count), and retaining PBTs from canonical repos plus any PBTs unique to forks (identified by SHA-256 of whitespace-normalized code). After fork deduplication the corpus contains 11,039 PBTs across 333 repositories from 281 unique GitHub users.

¹The dataset will be released on HuggingFace upon de-anonymization.

²The dependency-graph crawl was run once offline; the resulting `dependents.json` snapshot is read by the main scraper at startup.

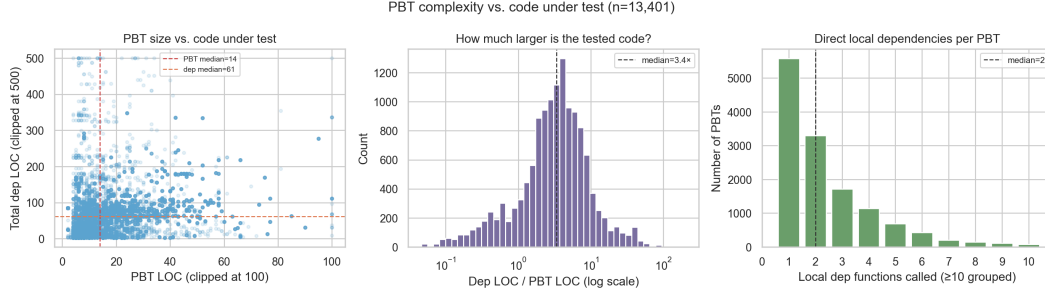


Figure 2: PBT complexity versus the complexity of the code each PBT tests, measured across 26,642 PBTs with recoverable local dependencies. *Left*: scatter of PBT LOC against total local-dependency LOC. *Centre*: distribution of the dep/PBT LOC ratio (log scale). *Right*: distribution of local project functions called directly per PBT.

4.2 Corpus statistics

Table 1 summarizes the post-deduplication input corpus; the full post-production pipeline that produces the final benchmark from these PBTs is described in Section 3. The star and fork distributions are heavily right-skewed: the median repository has 0 stars, reflecting that most real-world Hypothesis users work on small or personal projects. The median PBT is 13 lines with a cyclomatic complexity of 2, consistent with the concise, assertion-focused style typical of Hypothesis tests. Halstead effort, a measure of the cognitive work required to write the code, has a median of 3 and a long right tail (mean 207, 95th percentile 981), consistent with a corpus dominated by short, simple tests alongside a minority of complex multi-strategy properties.

Figure 2 characterizes the relationship between PBT complexity and the complexity of the code each PBT tests. On average, PBTs are concise (median 18 LOC) while their direct local dependencies total roughly 30× more code (mean 553 dep LOC). The ratio is right-skewed: many PBTs exercise a handful of short helper functions, but a substantial tail tests deeply nested call graphs spanning hundreds of lines. Notably, PBT complexity is nearly independent of dependency complexity (Pearson $r = 0.06$ between test cyclomatic complexity and dependency count), confirming that Hypothesis encourages engineers to write simple, focused tests regardless of how complex the underlying code is.

Comparison with the Hypothesis Corpus. DeVoe DeVoe [2026] recently released the Hypothesis Corpus 2026 (HC-2026): 28,928 Hypothesis tests from 1,504 repositories, assembled by the Hypothesis development team to study PBT *runtime behaviour*—how generators consume entropy, how tests are distributed in time, which predicates fire. We built RealPBT for a different purpose: supplying clean, license-clear, dependency-annotated PBTs for Lean formalization. These divergent goals drive the key design differences summarised in Table 2: HC-2026 is richer in runtime context (timing, coverage, entropy), while RealPBT provides what formal verification requires (dependency source, static metrics, NL summaries) with a stricter license filter. Table 2 also shows the repository overlap we computed programmatically: 28 exact matches after their validity filter, 111 (33%) before it. A full methodological comparison is given in Section 7.

5 Results

We present a characterization of the FVSpec benchmark along six axes: dataset scale, source code properties, translation quality, output complexity, pipeline cost, and difficulty stratification. All quantitative claims in this section correspond to programmatically generated figures; we report the dataset-level statistics from the public HuggingFace release and direct the reader to the accompanying visualizations for distributional detail.

Table 1: Summary statistics for the post-deduplication RealPBT input corpus.

Metric	Value
PBTs (pre-dedup, HuggingFace)	54,345
PBTs (post-dedup, working set)	11,039
Unique repositories	333
Unique GitHub users	281
Median PBTs per repo	6
Median repo stars	0
Mean repo stars	39
Median PBT LOC	13
Median cyclomatic complexity	2.0
Median Halstead effort	3
MIT-licensed repos	148
Apache-2.0-licensed repos	66
BSD-3-Clause-licensed repos	25

Table 2: Comparison of the FVSpec RealPBT corpus with the Hypothesis Corpus 2026 DeVoe [2026]. Both datasets deduplicate repos; only ours deduplicates at the test level.

	Ours	HC-2026
Collection	dep. graph	API search
# PBTs (pre-dedup)	54,345	~28,928 ^a
# PBTs (post-dedup)	11,039	23,139
# Repos	333	1,504
Repo dedup	fork/name	MinHash
Test dedup	SHA-256	none
License filter	perm. only	none
Dep. source code	✓	×
Code metrics	✓	×
NL summary	✓	✓
@settings	×	✓
Runtime data	×	✓
Coverage	×	✓
Repo overlap (valid)	28 / 333 (8%)	
Repo overlap (all)	111 / 333 (33%)	

^a 28,928 canonical (parametrization-deduped) nodes per their schema; 23,139 remain after code-level deduplication.

291 5.1 Dataset Scale

292 The released dataset contains 9,415 Lean 4 formalization challenges derived from 2,772 unique
 293 Python property-based tests. Two pipeline runs contribute to the dataset: an initial run (feb03, 5,979
 294 samples, Claude Sonnet) and a second run (apr08, 3,436 samples, Claude Sonnet 4.6). Because
 295 the same PBT may be formalized in multiple ways across runs, each unique PBT has between 1
 296 and 15 formalizations. Sixty-five percent of multi-formalization groups exhibit a Pareto front larger
 297 than one: no single formalization dominates on all structural faithfulness sub-metrics simultaneously,
 298 confirming that the multiple-formalization design captures genuinely complementary aspects of each
 299 PBT.

300 5.2 Python Source Characteristics

301 Figure 3 characterizes the input PBTs along three axes: lines of PBT code, average cyclomatic
 302 complexity (Radon), and maintainability index. The source tests span a wide range of complexity,
 303 from trivial single-assertion tests to multi-strategy tests exercising complex library APIs. This
 304 diversity is inherited directly from real-world engineering practice and is not curated or filtered for
 305 difficulty.

306 5.3 Translation Quality

307 We evaluate translation quality using *structural faithfulness*, a composite metric that decomposes
 308 into five sub-scores: parameter coverage, type correspondence, strategy coverage, assertion coverage,
 309 and dependency coverage (described in the preceding section). Figure 4 shows the distribution of the
 310 overall score and the mean of each sub-score across the dataset. The overall faithfulness distribution
 311 exhibits a clear mode above 0.5, indicating that the majority of translations preserve the essential
 312 structure of the source PBT, though a long tail of lower-quality translations remains.

313 Of the 9,415 samples, 59% reached a provided implementation level (full Lean code generated by the
 314 agent); the remaining 41% fell back to signature-only stubs. Structural faithfulness is comparable
 315 across both levels (mean 0.67 vs. 0.68), suggesting specification quality is largely independent of
 316 implementation completeness.

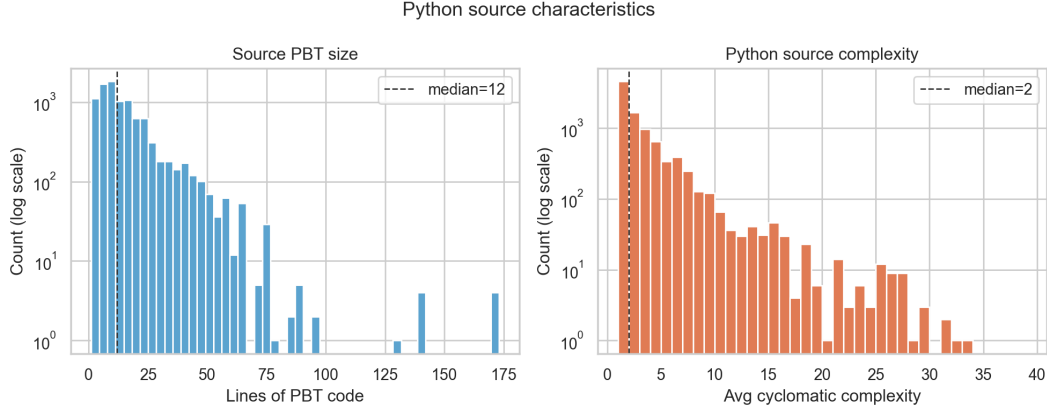


Figure 3: Python source characteristics of the input PBTs: lines of code, cyclomatic complexity (Radon), and maintainability index.

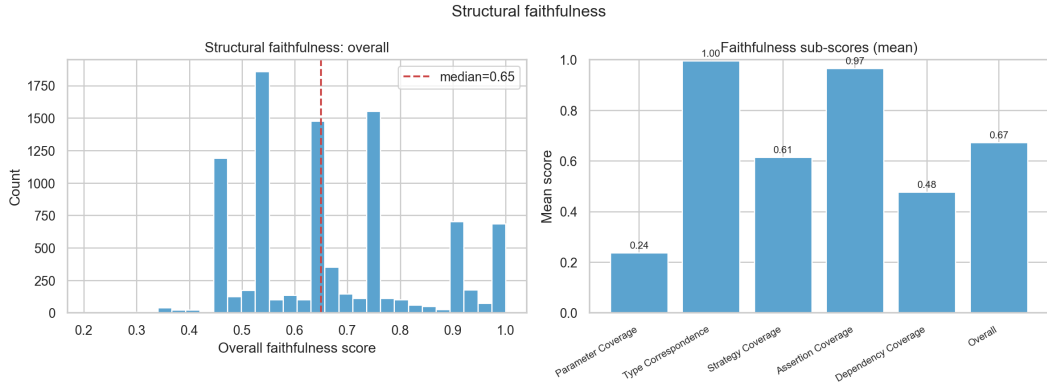


Figure 4: Structural faithfulness: distribution of the overall composite score (left) and mean sub-scores (right).

5.4 Lean Output Complexity

Figure 6 characterizes the generated Lean artifacts: total lines of Lean code, number of theorems per sample, and number of sorry placeholders. The median sample contains multiple theorems, each requiring an independent proof. The sorry count tracks the theorem count closely, as expected from our pipeline design (each theorem is emitted with a sorry placeholder).

5.5 Pipeline Cost

Figure 7 reports the per-sample pipeline cost in three dimensions: wall-clock time, token usage, and number of agent turns. The distributions are right-skewed, reflecting the iterative repair loop: most samples compile within a few turns, but a tail of difficult translations requires many repair iterations before the Lean LSP reports success or the budget is exhausted.

5.6 Difficulty Stratification

Each sample is graded for proof difficulty by Claude Haiku 4.5, which assigns a binary label (*easy* or *hard*) along with a confidence score. Figure 8 shows the resulting split and the distribution of grader confidence. The benchmark contains a substantial proportion of both easy and hard problems, and the grader expresses high confidence for the majority of assessments.

Figure 9 examines the relationship between difficulty and two output characteristics: structural faithfulness and Lean output size. Hard problems tend to produce longer Lean outputs, consistent

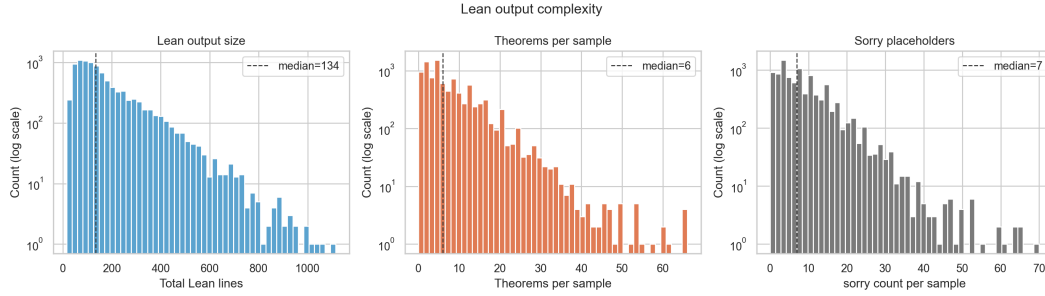


Figure 5: Lean output complexity: total lines (left), theorems per sample (center), and sorry placeholders per sample (right).

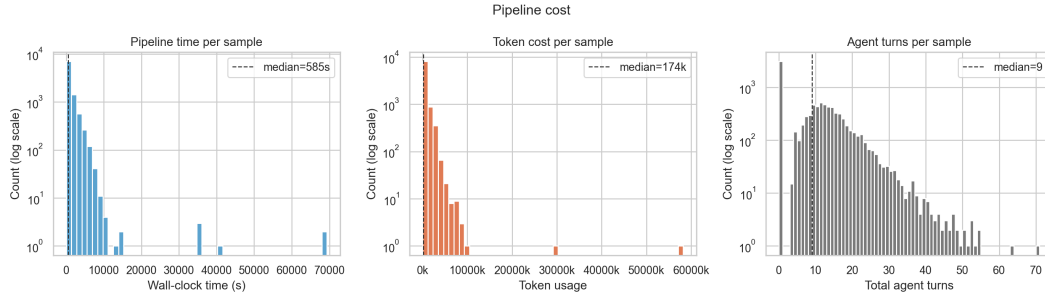


Figure 6: Pipeline cost per sample: wall-clock time (left), token usage (center), and agent turns (right).

334 with the intuition that more complex source PBTs lead to more elaborate formalizations and more
 335 challenging proofs.

336 5.7 Baselines

337 **TODO** Baseline evaluations are in progress. We evaluate four frontier models—Claude Sonnet 4.6,
 338 Claude Opus 4.6, GPT-5.4-mini, and Gemini 2.5 Pro—on a stratified sample of 75 problems (50%
 339 easy, 50% hard). Each model is given access to the Lean LSP via MCP tools and scored on two
 340 metrics: a binary *proved* score (zero sorry remaining and lake build succeeds with no sorry
 341 warnings) and *partial credit* (fraction of sorry placeholders removed, clamped to $[0, 1]$). Results
 342 will be reported as per-bucket (easy/hard) and aggregate prove rates, with a comparison table across
 343 models.

344 6 Threats to Validity

345 We note that the theorems we construct may be false—this is desirable because the AI has the
 346 opportunity to contradict the theorem (and perhaps identify a counter-example). Because the Lean
 347 theorem prover is a so-called foundational tool, once a theorem is verified or contradicted, we have
 348 near-perfect confidence the answer is correct. However, because our specifications are translated rather
 349 than hand-written, some may not faithfully represent the original PBT. Our structural faithfulness
 350 metric provides an objective measure of translation quality, but it is a heuristic: a high score does not
 351 guarantee semantic equivalence, and a low score does not necessarily indicate a useless problem.

352 Our benchmark does not include human baselines. We expect the dataset to exhibit a wide range of
 353 difficulty, including many problems that are trivial for current AIs, and many harder problems, includ-
 354 ing some insurmountable even to world-expert human teams. Without human expert performance
 355 data, we cannot directly calibrate this difficulty distribution.

356 The pipeline’s reliance on LLM-driven translation means that benchmark quality is bounded by
 357 the capabilities of the translation model. Samples where function discovery failed (8%) produce

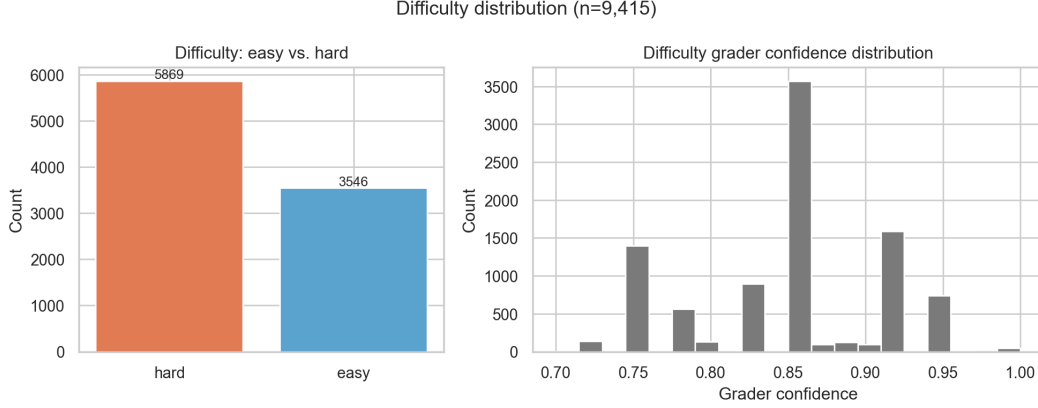


Figure 7: Difficulty distribution: easy vs. hard counts (left) and grader confidence distribution (right). n shown in title.

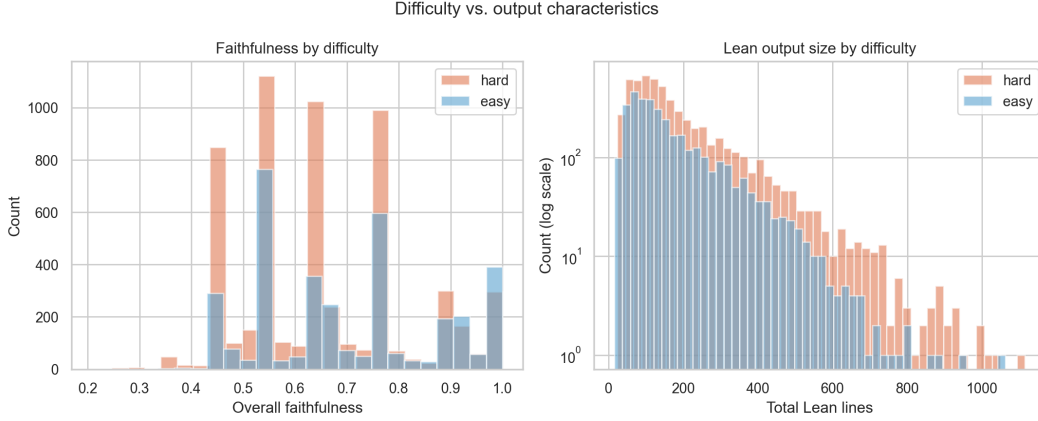


Figure 8: Difficulty vs. output characteristics: overall faithfulness (left) and total Lean lines (right), stratified by difficulty.

358 stub-based theorems that may be less representative. We mitigate this through postproduction filtering
 359 and the structural faithfulness metric, but some lower-quality samples may remain.

360 7 Related Work

361 Our work extends FVAPPS by Dougherty and Mehta [2025], which pioneered
 362 AI-driven translation of coding problems into Lean theorems. In FVAPPS, source programs were
 363 taken from the APPS benchmark Hendrycks et al. [2021], translated to PBTs, then lifted to theorems.
 364 We apply the same process at scale to PBTs found in the wild, rather than synthetic coding challenges.

365 **Lean verification benchmarks.** A growing number of benchmarks target Lean specifically. mini-
 366 CodeProps Brandfonbrener et al. [2024] provides 177 program specifications translated from TIP
 367 (Tons of Inductive Programs), finding that current LLM provers fail on medium and hard prop-
 368 erties. CLEVER Materzok et al. [2025] offers 161 curated problems sourced from HumanEval,
 369 testing end-to-end verified code generation. Verina Wu et al. [2025b] covers all three foundational
 370 tasks—code generation, specification generation, and proof generation—across 189 manually curated
 371 coding tasks. VeriBench Stechly et al. [2025] translates Python code with documentation into Lean
 372 implementations, specifications, and unit tests (113 tasks). The Vericoding Benchmark Bursuc et al.
 373 [2025] supports Lean, Rust/Verus, and Dafny, reporting success rates from 27% (Lean) to 82%
 374 (Dafny). VeriEquivBench Ma et al. [2025] uses LeetCode transformations to evaluate verifiable
 375 agents on novel data without contamination. These benchmarks are valuable but share a common

376 limitation: their specifications derive from synthetic coding problems or pedagogical exercises, not
377 from properties written by practicing engineers.

378 **Other verification tool benchmarks.** DafnyBench Loughridge et al. [2024] provides Dafny
379 verification tasks. VerusBench Chen et al. [2024] collects 150 proof tasks spanning Dafny, SV-COMP,
380 and Verus. SV-COMP Beyer [2025] is the standard competition for software verifiers, with over
381 33,000 C tasks, but targets automated verification rather than interactive theorem proving. InvBench
382 Wu et al. [2025a] focuses specifically on loop invariant synthesis. Chakraborty et al. Chakraborty
383 et al. [2025] explore neural synthesis for SMT-assisted proof-oriented programming in F*.

384 **Interactive theorem proving benchmarks.** LeanDojo Benchmark 4 Yang et al. [2024] provides
385 122,517 theorems from Mathlib4. CoqStoq Thompson et al. [2025] collects 196,929 theorems from
386 2,226 GitHub Coq repositories. These are primarily mathematical rather than software-oriented, and
387 their theorems exist in the training data of most frontier models.

388 **Mathematics benchmarks.** FrontierMath Glazer et al. [2024] evaluates advanced mathematical
389 reasoning. While formal verification and mathematics share proof infrastructure, the skills required
390 differ substantially: software verification demands modelling of program semantics, library APIs,
391 and computational behavior, rather than abstract mathematical reasoning.

392 FVSpec differs from existing benchmarks in three ways. First, our specifications originate from
393 real-world property-based tests written by practicing engineers, not synthetic problems or pedagogical
394 exercises. Second, the source PBTs were never formally verified, so most theorems in our benchmark
395 will not appear in any model’s training data. Third, we operate at a larger scale than existing Lean
396 verification benchmarks, with thousands rather than hundreds of problems. Our work is motivated by
397 the broader context of AI safety proposals premised on AI-driven formal verification, including the
398 Guaranteed Safe AI agenda Dalrymple et al. [2024].

399 **The Hypothesis Corpus.** As discussed in Section 4, our RealPBT corpus and HC-2026 DeVoe
400 [2026] share a target library but differ in purpose, collection, and deduplication methodology. HC-
401 2026 discovers repositories via API search for `import hypothesis` and retains only those whose
402 test suite installs under `pytest -collect-only`; we crawl the GitHub dependency graph and filter
403 by permissive license. They deduplicate at the repository level using MinHash ($J > 0.75$ across
404 30% of file pairs) but not at the test level; we do both. HC-2026 is richer in runtime context while
405 RealPBT provides dependency source code and static metrics. The two corpora are complementary;
406 future work could join them on the 28 shared repositories to pair dynamic behaviour with formal
407 specifications.

408 8 Conclusion / Future Work

409 Several extensions are possible:

410 **TODO:** Add examples from other languages with significant usage of PBT, e.g. Haskell/QuickCheck,
411 C++/RapidCheck. Translate our benchmark to other theorem proving systems, e.g. Coq, Isabelle,
412 F*, etc. More advanced website features, such as prediction markets about when the benchmark
413 will be saturated, and a pipeline for submitting code to the leaderboard which we then persist into a
414 training or SFT dataset. Our benchmark will also be useful for developing new techniques in program
415 repair and synthesis. If a property is disproven, this creates a new interesting challenge, namely to
416 synthesize a patch to the original function such that the theorem now holds. We hypothesize that
417 many such bugs will be particularly subtle and interesting edge cases since, by definition, in most
418 cases they were not detected by the original developer’s PBT harness.

419 References

- 420 D. Beyer. State of the art in software verification and witness validation: SV-COMP 2025. In
421 *Proceedings of the 31st International Conference on Tools and Algorithms for the Construction*
422 *and Analysis of Systems (TACAS)*, 2025.
- 423 D. Brandfonbrener, L. Beurerkellner, D. Romero, N. Amin, and M. Vechev. miniCodeProps: a
424 minimal benchmark for proving code properties. *arXiv preprint arXiv:2406.11915*, 2024.

425 S. Bursuc, T. Ehrenborg, S. Lin, L. Astefanoaei, I. E. Chiosa, J. Kukovec, A. Singh, O. Butterley,
426 A. Bizid, Q. Dougherty, M. Zhao, M. Tan, and M. Tegmark. A benchmark for vericoding: formally
427 verified program synthesis, 2025. URL <https://arxiv.org/abs/2509.22908>.

428 S. Chakraborty, G. Ebner, S. Bhat, S. Fakhoury, S. Fatima, S. Lahiri, and N. Swamy. Towards neural
429 synthesis for SMT-assisted proof-oriented programming. In *Proceedings of the 47th International
430 Conference on Software Engineering (ICSE)*, 2025.

431 H. Chen et al. VerusBench: A benchmark for automated verification of Rust with Verus. *arXiv
432 preprint arXiv:2409.13082*, 2024.

433 D. Dalrymple, J. Skalse, Y. Bengio, S. Russell, M. Tegmark, S. Seshia, S. Omohundro, C. Szegedy,
434 B. Goldhaber, N. Ammann, A. Abate, J. Halpern, C. Barrett, D. Zhao, T. Zhi-Xuan, J. Wing, and
435 J. Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI
436 systems. *arXiv preprint arXiv:2405.06624*, 2024.

437 L. DeVoe. The Hypothesis corpus. [https://hypothesis.works/articles/
438 hypothesis-corpus/](https://hypothesis.works/articles/hypothesis-corpus/), 2026. Accessed April 2026.

439 M. Dodds. Formally verifying industry cryptography. *IEEE Security & Privacy*, 20(3):65–70, 2022.
440 doi: 10.1109/MSEC.2022.3153035.

441 Q. Dougherty and R. Mehta. Proving the coding interview: A benchmark for formally verified code
442 generation. In *Proceedings of the LLM4Code Workshop at the 47th International Conference on
443 Software Engineering (ICSE)*, Ottawa, Ontario, Canada, 2025. doi: 10.48550/arXiv.2502.05714.

444 E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, C. Falkman Olsson, J.-S.
445 Denain, A. Ho, E. de Oliveira Santos, O. Järvinemi, M. Barnett, R. Sandler, M. Vrzala, J. Sevilla,
446 Q. Ren, E. Pratt, L. Levine, G. Barkley, N. Stewart, B. Grechuk, T. Grechuk, S. Varma Enugandla,
447 and M. Wildon. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in
448 AI. *arXiv preprint arXiv:2411.04872*, 2024.

449 D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He,
450 D. Song, and J. Steinhardt. Measuring coding challenge competence with APPS. In *Proceedings
451 of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.

452 HypothesisWorks. Dependents of Hypothesis on GitHub. [https://github.com/
453 HypothesisWorks/hypothesis/network/dependents](https://github.com/HypothesisWorks/hypothesis/network/dependents), 2024. Accessed 2024.

454 C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu,
455 N. Amin, and M. Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv
456 preprint arXiv:2406.08467*, 2024.

457 X. Ma et al. VeriEquivBench: Evaluating verifiable agents on novel data without contamination.
458 *arXiv preprint arXiv:2510.06296*, 2025.

459 D. R. MacIver, Z. Hatfield-Dodds, and many other contributors. Hypothesis: A new approach to
460 property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.
461 01891.

462 M. Materzok, W. Szulc, M. Tabiś, J. Kozak, and B. Piotrowski. CLEVER: A benchmark for
463 closed-loop end-to-end formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.

464 K. Stechly et al. VeriBench: Benchmarking end-to-end formal verification. In *AI4Math Workshop at
465 ICML*, 2025. URL <https://openreview.net/forum?id=rWkGFmnSN1>.

466 K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. F. Ferreira, S. Lerner,
467 and E. First. Rango: Adaptive retrieval-augmented proving for automated software verification,
468 2025. URL <https://arxiv.org/abs/2412.14063>.

469 T.-Y. Wu et al. InvBench: A benchmark for loop invariant synthesis. *arXiv preprint arXiv:2509.21629*,
470 2025a.

471 W.-D. Wu, K. Wang, R. Mangal, and I. Dillig. Verina: Benchmarking formally verified code
472 generation. *arXiv preprint arXiv:2505.23135*, 2025b.

473 K. Yang, A. Swope, A. Gu, R. Chalapathy, P. Song, S. Lember, A. Thomson, and A. Anandkumar.
474 LeanDojo: Theorem proving with retrieval-augmented language models. *Advances in Neural*
475 *Information Processing Systems (NeurIPS)*, 2024.